ASSIGNMENT 2—Extend your example-tracing tutor

05-432/05-832 Personalized Online Learning, Fall 2026
Due: Sun, September 13, 5pm, on Canvas and on Tutorshop

In this homework, you are asked to improve your dose calculation tutor from Assignment 1, by adding tutor functionality and extending the tutor's problem set, as well as to test your tutor with two users. You are also asked to address the feedback you received on your submission for Assignment 1. Key goals in the homework are to learn about advanced forms of ITS tutor behaviors (supported by CTAT) and to continue to think about design issues for ITSs. As well, the experience of seeing your tutor in the hands of users is always illuminating!

Grading Rubric

Grading criteria include:

- Does your tutor function correctly?
  - Does it support 3 problems made through Mass Production?
  - Does the tutor do some of the arithmetic computations for the student, either through input substitution or tutor-performed actions?
  - Does the tutor use formulas?
  - Is the tutor flexible in terms of different solution paths? That is, does it accept at least two solution paths within each problem?[1]
  - Is the tutor flexible in terms of the step order? That is, is the step order constrained in a way that is reasonable and helpful for the student, but minimal way (i.e., not more constrained than strictly needed)?
  - Does the tutor address the feedback on Assignment 1?
  - Did you test the tutor with two users, and based on what you found, generate ideas for how the tutor might be improved further?
- Did you create a problem set on Tutorshop?
- Does the report include the required elements, described below?

Start with a New Package by Copying Your Assignment 1 Package

To preserve your Assignment 1 submission, start with a new package for Assignment 2. A good first step for this assignment is to make a copy of your Assignment 1 package.

Make the Tutor Do the Arithmetic for the Student

If we consider practicing decimal arithmetic to be outside of the learning objectives for the dosage calculation tutor (we don't expect vet techs to do complex decimal arithmetic in their heads), then it would make sense that the tutor takes care of the arithmetic calculations for the student – that is, that some of its input fields would function as a calculator. Doing so would reduce cognitive load for the student, meaning you would be following one of the Tutor Interface Design principles. Having input fields with calculation capabilities is a common design pattern in math tutors, when the focus is not

[1] If you believe your tutor's interface does not allow for multiple solution paths, please talk to me in class. I will of course not require multiple paths when there aren't any.

on learning the given types of calculations. Note: In tutor units focused on helping students learn arithmetic skills, these calculations would constitute over-scaffolding.

For the main steps in your tutor where calculations happen, make your tutor do the calculations. Also, where the tutor interface requires merely copying numbers, make your tutor do the copying, unless, again, when this would mean taking over a reasoning step that the student must learn (e.g., when they must learn *which number to copy)*. You can create calculator functionality in a tutor in two different ways: You can either use *tutor-performed actions* (TPAs) or use *input substitution*. Also, the use of Formulas may come in handy -- and is in fact required for this assignment – see the next section. Information about the use of tutor-performed actions in CTAT can be found in OLI Module 3, Record Problem-Solving Behavior and
https://github.com/CMUCTAT/CTAT/wiki/Dynamic-Interfaces
Making input fields function as calculators has two steps:

1. In the Input Matcher for the Input, create a formula that checks that the student's input expression evaluates to the right number; use a Formula Match with formula algEval(input) and operator "=".
2. In the Input Substitution for the given link, replace the input with the formula algEval(input).

For more information about input substitution and TPAs, see the CTAT FAQ file in the student folder on the Google Drive.

Use Formulas

You are asked to use formulas (and/or regular expressions) in at least two steps in the tutor and in two hints. You may use formulas, for example, to make your tutor do calculations in certain input fields, as described in the previous section. For example, algEval(input) is a formula and thus (when used e.g., in input substitution) would count towards the requirement of using formulas in two steps. Regular expressions can be helpful in generalizing the input accepted on a link. You may use regular expressions, for example, to capture small notational variations in input, such as different ways of expressing units in steps (e.g., "1.6 lb" v. "1.6lb" v. "1.6 lbs" v. "i.6lbs"). See OLI Module 3, Generalizing a Behavior Graph.

For documentation, see https://github.com/CMUCTAT/CTAT/wiki/Using-Formulas and https://github.com/CMUCTAT/CTAT/wiki/Formula-Functions. Also, in your FractionAdditionSample package, see "07-1416-formulas.brd." (Launch together with The left branch of the behavior graph (which represents solutions with any common denominator greater than 12) has many formulas, both in the Matcher Settings for the inputs, and the hints. E.g., try solving the problem with common denominator 36 or some other number greater than 12. Study how the hints will adjust to the common denominator you decide to use. Study the formulas on the behavior graph links that make this possible. Note: This is one of the ways in which a behavior graph can stand for a wider range of behavior than only (literally) the demonstrated steps in the demonstrated order.

Tips regarding the use of Formulas

To use a formula or regular expression to define the range of accepted inputs for a given link (including making the input depend on the values accepted by the tutor on other

links), you can set the Input Matcher to "Formula Match," select one of the comparison options that appear below the Matcher Settings box, and express a formula in CTAT's formula language. To embed a Formula in a hint, use the following notation: <%=myFormula%> (i.e., "<%=" before and "%>" after the formula – without the double quotes. The value of the formula will be shown in the hint.

Formulas can reference variables! Specifically, they can use values accepted (by the tutor) in other input fields in the tutor, which is often helpful. The formula may need to check that these variables have a value, especially when the tutor allows flexibility in the order of steps. (If the step has not been entered yet by the student, the variable will not have a value.) For example, assume you have two input fields named inp1 and inp2, respectively. Assume further that in a third input field, you would like to accept, as input, the sum of the values of the inputs accepted in inp1 and inp2. For the Input Matcher, you might write the following formula (with the "=" option in the Link Editor):

```
inp1 + inp2
```

Alternatively, you might write the following, more elaborate, formula (with the "Boolean" option):

```
hasValue(inp1) && hasValue(inp2) && input == (inp1 + inp2)
```

The second version is better. The first one often works, but not when either inp1 or inp2 does not have a value yet; in that case (the tutor using) this formula will accept, as correct, student input "NaN" (which signals that the formula evaluation failed), which is undesirable, even if students are not likely to enter that input. The second version avoids this problem. Testing that referred variables have values is more important when a formula is embedded in a hint. For example, in the bottom-out hint for your third step, you might use the following formula:

```
<%=ifThen(hasValue(inp1) && hasValue(inp2),
        "The value is "+(inp1 + inp2),
        "The value is not known yet, you need to do other steps first.")%>
```

If you just had

```
<%="The value is "+(inp1 + inp2)%>
```

the hint would say: "The value is null" when either of the input fields inp1 or inp2 would not have a value. The more elaborate formula avoids this problem.

Add a Solution Path

If your interface allows multiple ways of solving (e.g., by converting the animal's weight to pounds or to ounces, as a step towards converting it to metric units), add an additional solution path[2]. The path you add could for example be based on your findings from testing the tutor with users as part of Assignment 1.

[2] If you believe your tutor's interface does not allow for multiple solution paths, please talk to me in class. I will of course not require multiple paths when there aren't any.

Place Reasonable but Minimal Constraints on the Order of Steps

So as to place some reasonable limitations on the step order in your tutor, while still allowing for substantial flexibility, create at least two *groups* in your tutor. See OLI Module 3, Generalizing a Behavior Graph. For more fine-grained definitions of required step order (e.g., if *some*, but not *all*, steps should be done in order, as is often the case), you may create ordered or unordered groups of links within your behavior graph. See "05-1416-groups.brd" and "05-1416-groups-v2.brd" in the FractionAdditionSampler package. A student must follow the order of an unordered group but is free to do the steps of an unordered group in any order. Ordered and unordered groups can be nested within each other.

To create a group, select a link in the behavior graph, right-click, and select "Create new group." Give the group a short but meaningful name. To add a link to a group or remove a link from a group, right click on the link and select the corresponding command. Groups must be formed one link at a time. Groups can be nested, which is often useful.

Address Feedback from Assignment 1

Update your tutor to reflect the feedback you received on Assignment 1 from the course instructors.

Create New Problems using Mass Production, Including One Generated by an LLM

Once you have your tutor running, tested, and polished, you are asked to make three dose calculation problems using CTAT's Mass Production process. This process is documented below in this Assignment. See also OLI Module 3, Generating Isomorphic Problems, and https://github.com/CMUCTAT/CTAT/wiki/Mass-Production One of the three problems can be your original problem from Assignment 1, now brought under the Mass Production "regime." The second should be a new problem generated by you, though Mass Production. You may select it from the list below. The third must be a new problem, generated by an LLM, using CTAT's built-in facility for doing Mass Production with an LLM. We will discuss in class how you can use an LLM in Mass Production. Note: You can simply let the LLM generate whatever problem it sees fit, perhaps telling the LLM you want a common scenario at a vet's office, or perhaps specifying the animal. These choices are at your discretion. Please be sure to save your prompt used for the LLM in your package on Tutorshop (please see below for how to do so).

Since it is probably wise to not fully trust the LLM's output, check if the numbers and hints are correct. Also check that the LLM-generated problem is reasonable, for instance, whether the given drug is suitable for the given animal, and whether the animal's weight and the dosage given are within reasonable limits. You may use Gemini for that purpose or some other LLM or a traditional web search.

Create a Problem Set for Your Tutor on Tutorshop

Within your package, create a problem set with your three problems. Your .html and .brd should appear on your package page's **Assets** tab (on the package screen in Tutorshop). Then:

1. Use your package page's **Problems** tab to create problems: A CTAT *problem* is a paring of a model file (here, a .brd file) with a student interface (a .html file).
2. After you have created one or more problems, use your package page's **Problem Sets** tab to group problems into a problem set. You can run the tutors by clicking the green triangle button on your problem set's row in your package's problem set list.

Test your Tutor with Two Users

Once you tutor is ready, ask two users to try your tutor, specifically, users who do not know the task domain well (i.e., preferably not your classmates). Ask your participants to think out loud, take notes, interview them afterwards about what was clear and what was not, think about how you would redesign the tutor, based on your observations. It may be best to do this with the tutor uploaded on Tutorshop (though you could also run the tutor from the authoring tools). You are asked to reflect on how you could improve your tutor design, based on your observations.

What to Turn in

A. Please make sure your tutor, with a problem set containing at least three problems, is available on Tutorshop. No need to submit your tutor through Canvas.

B. Also, submit, through Canvas (to Assignments > Assignment 2), a report of no more than 3 double-spaced pages (hard limit – screenshots don't count against the page limit)
   1. The name of the package of your tutor on Tutorshop.
   2. Description of the new tutor features you added
      a. Describe how you made the tutor do calculations for the student (which steps, what calculations, how did you implement them?)
      b. Describe your use of Formulas (which steps, what do they do?)
      c. Describe what solution path you added.
      d. Describe how you use groups to constrain the order of steps, and why?
      e. Describe how you addressed the feedback from Assignment 1.
   3. Description of tutor testing
      a. As you tested the tutor with users, what went well and what did not go well?
      b. State redesign ideas for the main observed shortcomings of your tutor.
   4. Brief answers to the following questions:
      a. What was most difficult about doing this assignment? (Describe in 2-3 sentences.)
      b. How much time did you spend?
      c. How was the experience of using CTAT's LLM-based Mass Production feature?
      d. What did you like / didn't like about working with CTAT? What could we do to improve it?

Mass Production of Problems with CTAT

When you need to create many problems of the same type (as you typically when building an intelligent tutoring system), you can use CTAT's Mass Production process. This process analogous to the many mail merge utilities that are available. It relies on having a *behavior graph template* and a *problems table*; CTAT does the rest. It can generate new behavior graphs by drawing values from the problems table and substituting them into the behavior graph template, once for each new problem.

Once you have a single concrete (problem-specific) brd (i.e., not a template), you can apply Mass Production as follows:

First, turn your *concrete .brd* into a *.brd template* plus (the beginnings of) a *problems table* as follows:

1. In the CTAT Package List, right-click on the concrete .brd file that you want to turn into a template and choose Launch Mass Production: you should see your .brd file in the "Template File" field of the Mass Production wizard, as the file to be made into a template.
2. Run "Variablize All" – by clicking the eponymous button. This will create both the beginnings of a template and the beginnings of a (compatible) problems table, with one problem in the table. Some hand editing may still be needed. From the menu that pops up when you click "Variablize All," select the link attributes for which you would like to have variables in your template – i.e., the ones you expect will vary by problem for most (or even, all) of the links in the behavior graph. Just accepting the default set of attributes is usually fine. Click "Sync Selected." (Note: "Sync" and "Variablize All" mean the same thing – we are still working on standardizing the terminology.) This gives you a new folder in the Package List named "Mass Production" if you did not already have one, with two new files, a template and a problems table.
3. You will want to inspect the two files.
    a. You can open the template in CTAT's Behavior Graph Editor, just like any other .brd file, simply by double clicking on its name in the Package List. In the Graph window and in the Link Inspector, you will see many Mass Production variables with syntax %(nameOfTheVariable)%. Check out a few of the links. Scroll down in the SAI Matching tab; also check the Hints/Skills tab, to get an understanding of what the "Variablize All" process does.
    b. To inspect the problems table (.tsv file), drag it into the "Table File" slot in the Mass Production Wizard and lick "Open Table." A spreadsheet opens. It has two columns, one with the variables that have been created, the second with values for these variables for the problem you started with. The values in the cells in row 1 become the problems' names.
    c. Just for kicks, you can now check if CTAT's generate step, which fills out the problem-specific values into the template, gives you back the .brd that you started with. (It should of course do so –this is just to become familiar with the process.) Drag the new .brd template file into the "Template File" slot and the new problems table into the "Table File" slot. Specify in

which folder you want the new .brd to go by dragging one of the folders from your Package List (e.g., Mass Production) into the "Destination Folder" slot. Click "Generate BRDs." Check the resulting .brd, for example, by running it in Test Tutor mode.

4. The next step is the manual fine tuning of the template and problems table. The idea is that "Variablize All" will do the majority of the work but will likely generate unnecessary variables in a few places and will not insert variables in a few other places where they are needed. So you (as the author) need to identify these places and add or remove variables. In doing so, it is important to keep the template and the problems table compatible, meaning that they have the same set of variables. CTAT helps. Go through the template link-by-link (or perhaps go through the table row-by-row, or even, both) and then make changes as needed, for which (if you do it in the Link Editor) you can use the "Variablize" and "Unvariablize" buttons. These commands update not just the behavior graph template but also make corresponding changes in the given problems table.
5. When you are satisfied with the template and problems table, generate your final .brd (with the "Generate BRDs" command). Check that this .brd works fine in Test Tutor mode.

Second, once you are satisfied that your template and problems table are fine (i.e., capture all problem-specific information), generate more problems in the table. This can be done in two ways: by hand or by invoking CTAT's built-in LLM-based Mass Production Process. The latter is still a bit experimental, so we'd be grateful for your feedback as to how to improve it.

6. By hand: Add one or more columns in the table by hand. (Hint: Do so by copy-paste-edit applied to a previous column, but be sure to change the value in row 1 to specify a new problem filename.) Generate a new .brd or .brds, open them, inspect them, test them in Test Tutor mode.
7. The other option is to ask an LLM to generate more problems. For this purpose, CTAT's MP Wizard integrates the Claude foundation model. In the MP Wizard, click on the "Prompt LLM" tab. On the left, you will see a prompt for the LLM that you can customize. On the right, you can specify information that gets filled into the prompt to customize it. Fill out the fields on the right. Make sure an MP Table is specified (e.g., the one you were working on).
8. It may be wise to save the prompt, so you can edit it later.
9. Click "Generate." This process may take a while as it relies on an external server.
10. You will need to inspect the newly-created problems. You probably will want to do so both by inspecting the problems table *and* by generating .brds for the new problems and checking out the new problems in your running tutor. Our experience indicates that it is wise to check very carefully whether Claude's output is correct in all its details.

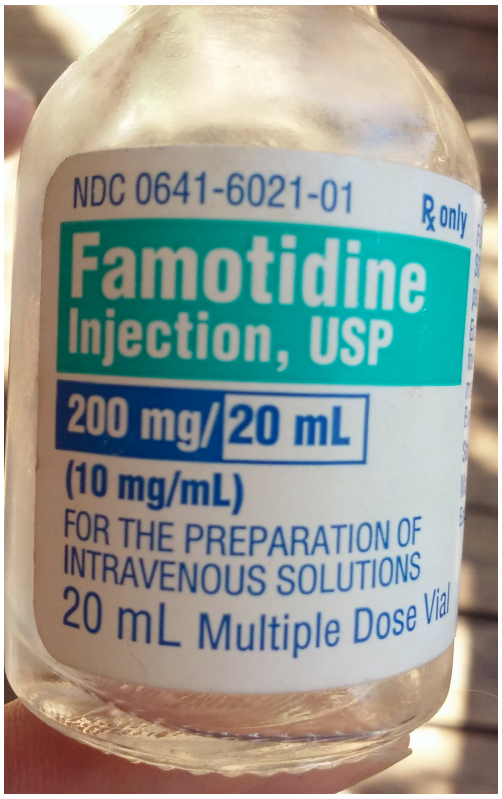
Dose Calculation Problems to Add to Your Tutor

Here are some additional tutor problems, from which you may select two, to be added to your tutor through Mass Production (see the instructions above). You may also create your own veterinary dose calculation problems.

Problem 1 (same as for Assignment 1)
The veterinarian wants to use Telazol® (tiletamine and zolazepam) to restrain a ferret for a short diagnostic procedure. The dosage is 22mg/kg IM. This ferret weighs.1lb, 10oz. What is the dose in milligrams and milliliters of the product shown to be used on this ferret? The Telazol solution has concentration 100mg/mL.

Problem 2
Zeke is a fairly chubby cat who weighs 11 pounds 5 ounces. He needs to be treated with Revolution, a flea treatment with the active ingredient selamectin. The dosage for a cat is 6mg of selamectin per kilogram. You have a vial of Revolution; the label says "2ml, 240 mg Selamectin." How much selamectin should Zeke get? How much of the liquid will you draw up in a syringe?

Problem 3



Skinny Cat is skinny; she only weighs 6 pounds 6 ounces. She has excess stomach acid and needs to be treated with famotidine,the active ingredient in PepsidAC. The bottle says 200mg per 20ml (see the photograph). The correct dosage is 0.5mg per kg of body weight. How much famotidine should Skinny Cat receive? How much of the liquid will you draw up in a syringe?

Problem 4
Fluffy the dog is prescribed prednisolone for his skin allergies. The veterinarian wants him to get 1mg/kg. Fluffy weighs in at 26 pounds and 7 3/4 ounces. The vet has a liquid

that contains 5mg/cc in the pharmacy cabinet. How much of the steroid does Fluffy get? And, how much cc does the owner give Fluffy every day?

Problem 5

Last Sunday, Steffi went to the shelter for an emergency foster check with four kittens because they had diarrhea and vomited. The veterinarian, when called during her time off on the weekend, gave instructions to give Metronidazole, 2x a day. When Steffi came home, she realized did not ask for the dose and checked the label of the Metronidazole bottle. It did not say, however, what dosage she was supposed to give! Since it was a holiday weekend, and the kittens needed their medication, she did her own calculations: The label said 80mg/ml, the kittens weighed 1 pound 9 ounces. Steffi checked the big red book that gives dosages for veterinarians: Metronidazole for cats with inflammatory gastrointestinal conditions is 10 mg/kg. How much of the antibiotic did each of the kittens get? How much of the liquid did Steffi squirt into their mouths?

Problem 6



Steffi currently has a nursing feral cat in foster care. This cat started to show signs of an upper respiratory infection, and Steffi decided to give her doxycycline, an antibiotic. Since the cat is nursing kittens, she did not want to be weighed, so Steffi had to estimate the cat's weight from the way she looks. She estimated that the cat weighs 7 pounds. Doxycycline is given at 10 mg/kg. How much of the antibiotic does the cat need? Steffi has a bottle of doxycycline solution that is 50mg/ml. How much of the (bitter-tasting) liquid does Steffi need to hide in the cat's tuna meal?

Problem 7

Banksy the cat was diagnosed with a urinary tract infection. The vet decided that the injectable antibiotic Convenia was the best choice. Banksy was not very cooperative and so an exact weight was not taken; his weight was estimated at 8 1/2 pounds. Convenia is given at 8mg/kg. The injectable solution contains 800 mg per 10ml. How much of the antibiotic did Banksy need? And how of the liquid solution did the veterinarian draw up?